PRÁCTICA 11

Ejecutar el contenedor del proyecto sin administrar servidores

Equipo	Los Rojos
Materia	Cómputo en la Nube
Unidad	Unidad II · Segundo parcial
Sesión	Miércoles 4 de noviembre de 2026
Objetivo	Desplegar la imagen del proyecto en ECS Fargate y exponerla mediante el balanceador de carga.
Integrantes	Flores Guevara Ángel Gabriel Kevin Ricardo López Payán
Docente	Elizabeth Gaxiola Carrillo

Introducción

Tener una imagen no basta: alguien tiene que ejecutarla, vigilar que siga viva y reponerla si muere. Un servicio administrado sin servidores hace ese trabajo y factura por los recursos que la tarea consume, sin que el equipo dimensione ni parchee máquina alguna. Esta práctica cierra el ciclo iniciado en la práctica 3: del servidor administrado a mano al contenedor que se administra solo.

1. Despliegue completo del servicio ECS Fargate

El script aws-practica11-ecs-fargate.sh ejecutó 13 pasos automáticos: recuperar recursos existentes, crear el clúster, NAT Gateway, security groups, roles IAM, task definition, target group, conectar el ALB y lanzar el servicio en subredes privadas.

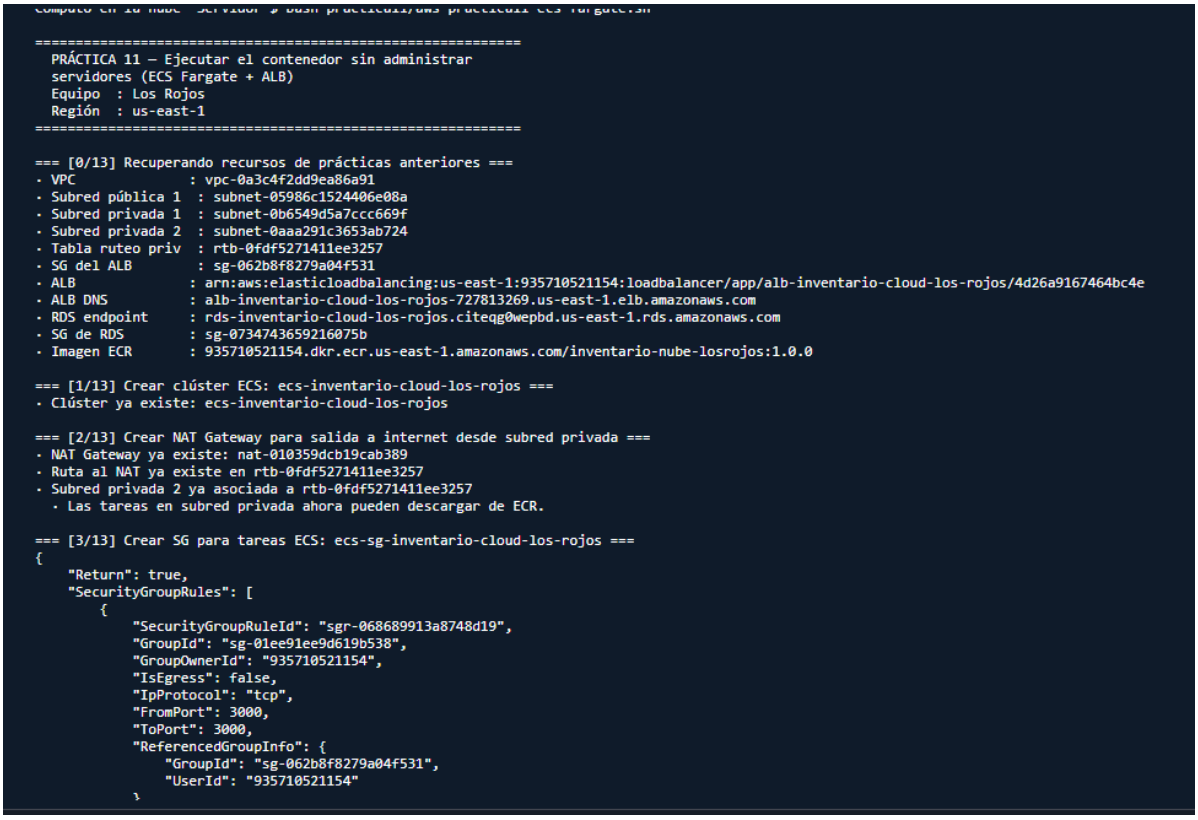


Figura: Pasos 0-3: recuperación de recursos (VPC, subredes, ALB, RDS, ECR) + clúster creado + NAT Gateway + SG ECS

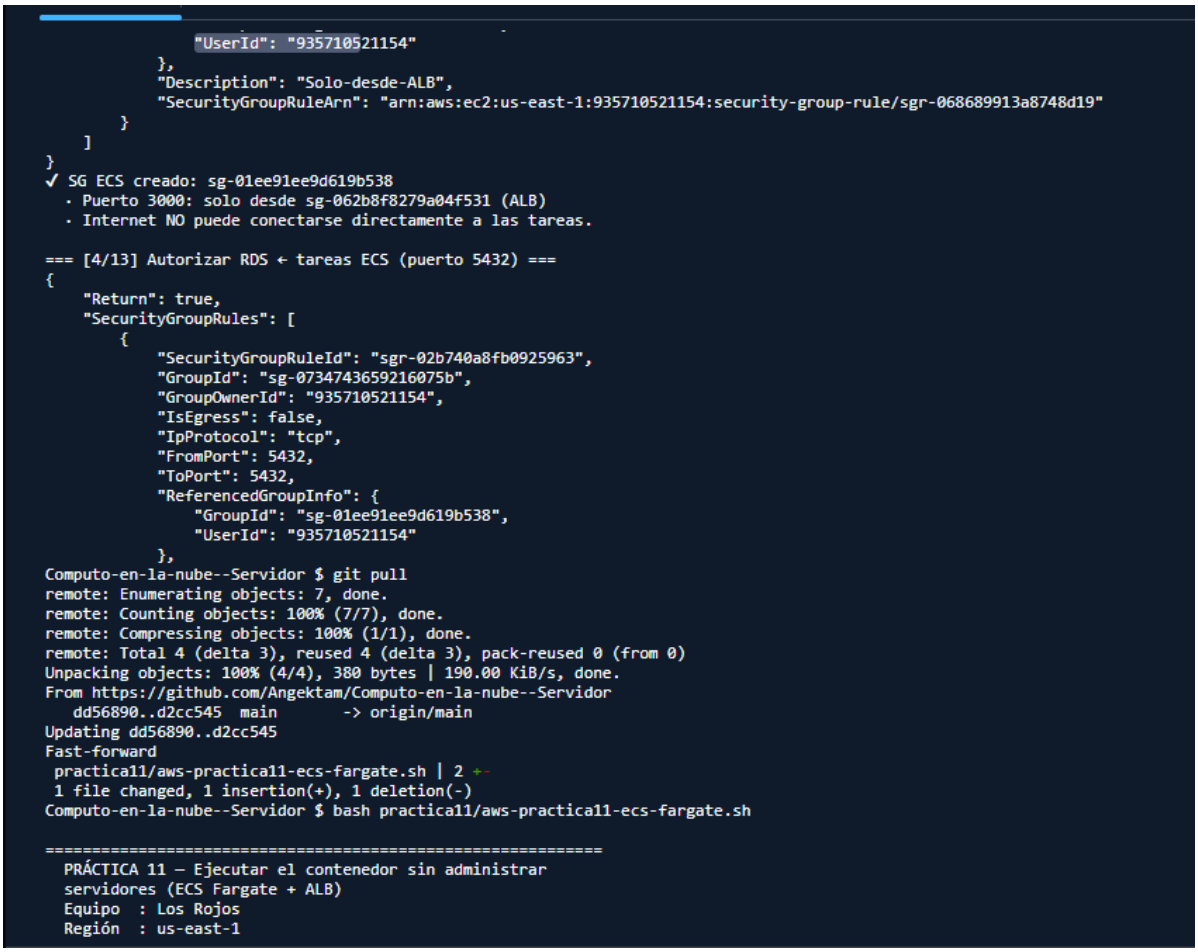


Figura: Pasos 3-4: SG ECS creado (puerto 3000 solo desde ALB) + RDS autorizado para recibir de las tareas ECS

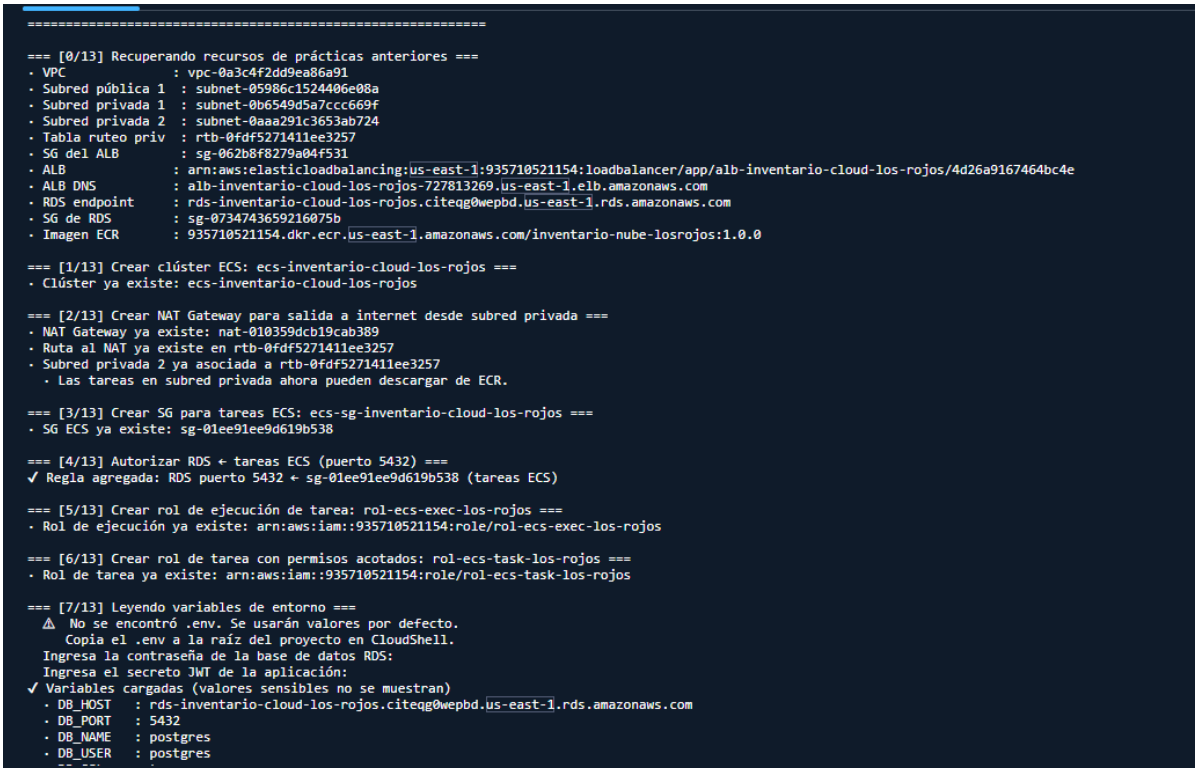


Figura: Pasos 5-8: roles IAM creados + variables de entorno cargadas + task definition registrada (imagen ECR, CPU 256, memoria 512, puerto 3000)

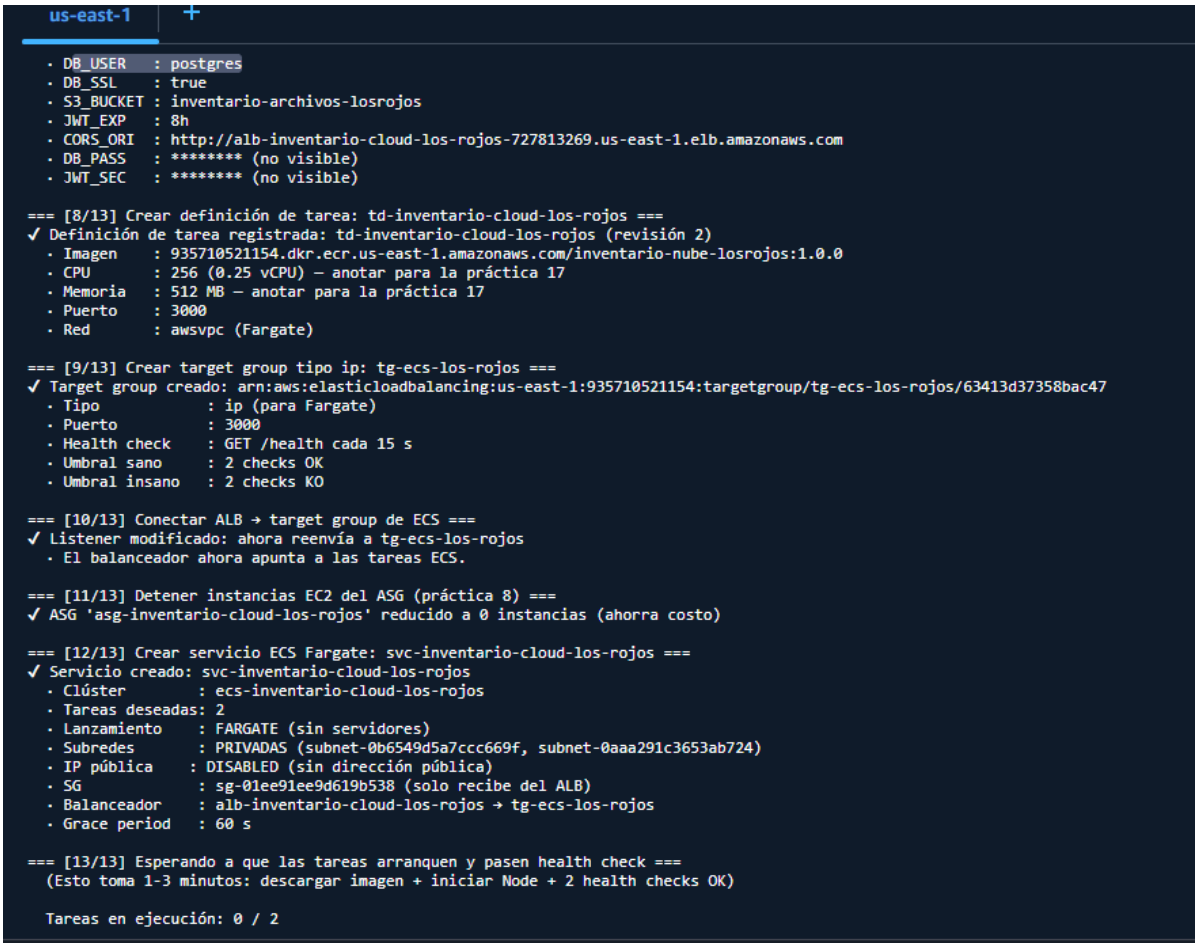


Figura: Paso 8 ampliado: variables sensibles enmascaradas, DB_HOST inyectado, target group tipo ip y ALB conectado al servicio ECS

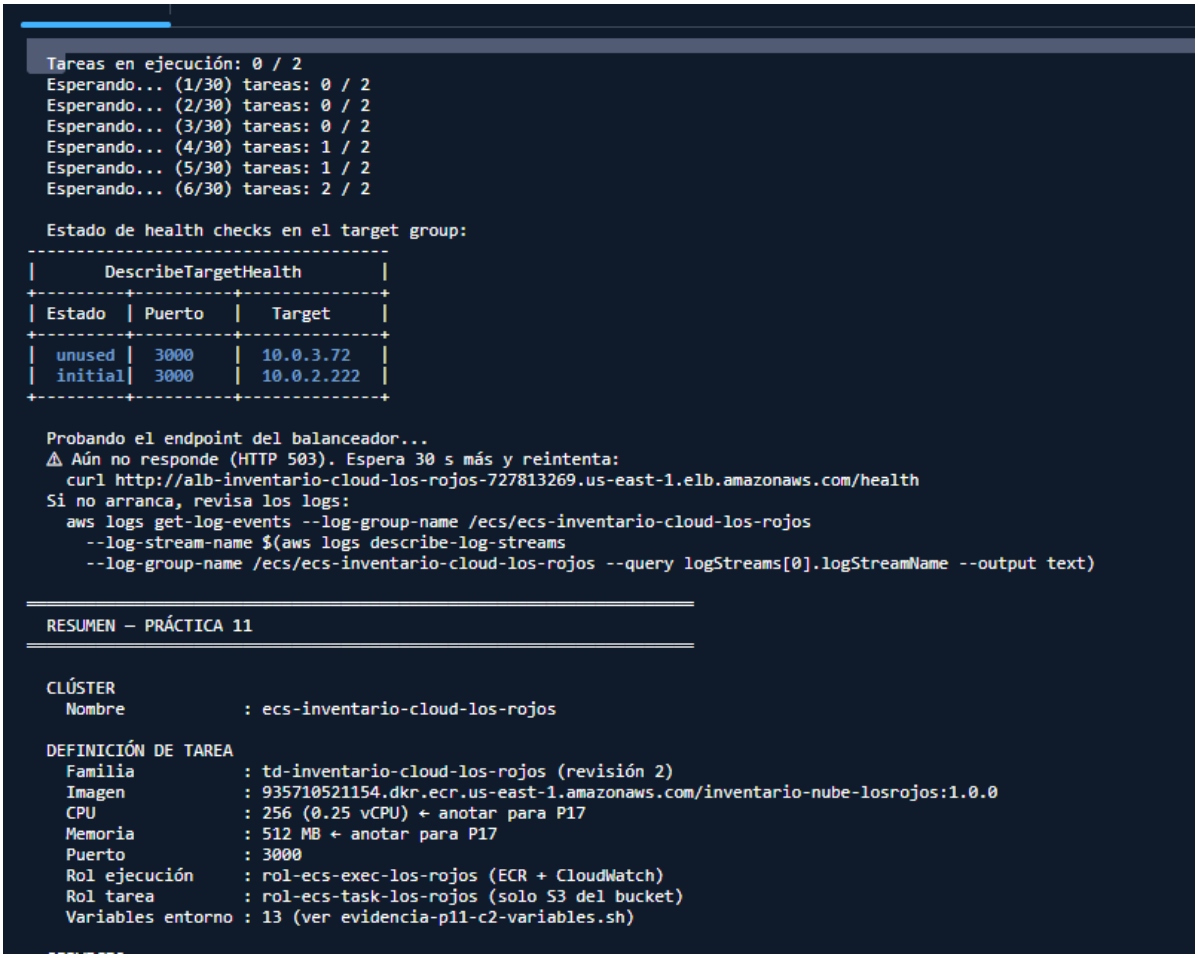


Figura: Pasos 9-13: target group creado, listener modificado, ASG reducido a 0, servicio Fargate creado en subredes privadas, tareas arrancando (2/2)

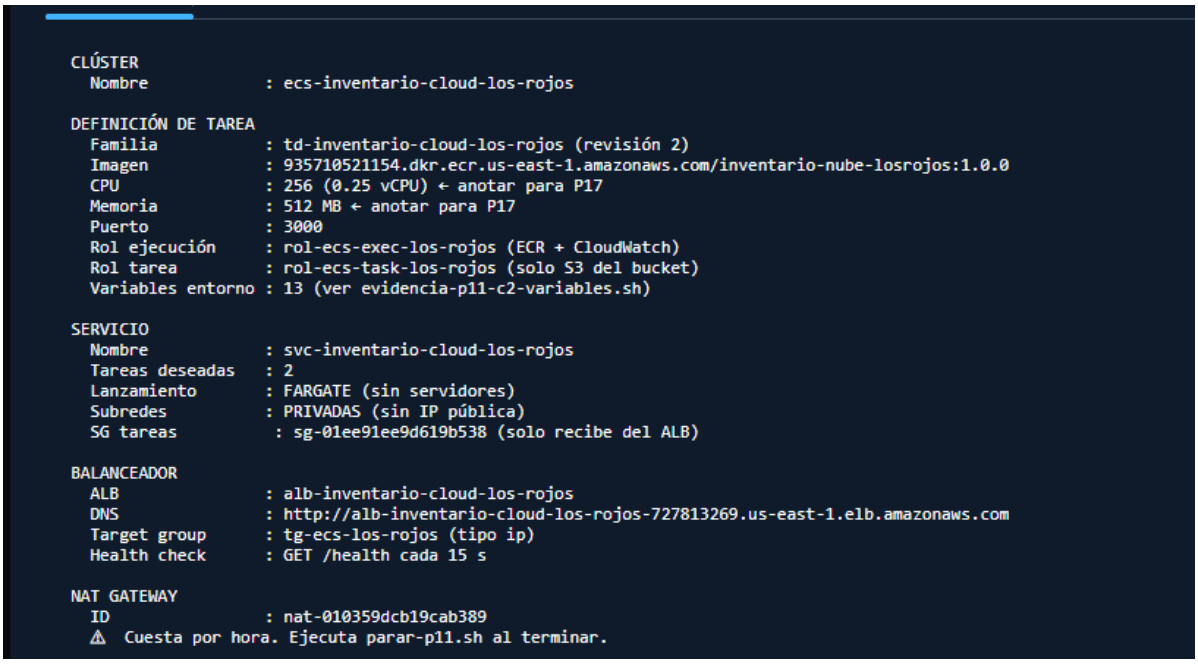


Figura: Resumen final del despliegue: clúster, task definition rev 2, servicio FARGATE con 2 tareas deseadas en subredes privadas, DNS del ALB

1.1 Ajuste de configuración: certificado SSL de RDS

Durante el despliegue se detectó que RDS usa un certificado SSL auto-firmado en cadena. Se agregó la variable DB_SSL_REJECT_UNAUTHORIZED=false a la task definition (revisión 3) y se actualizó el servicio. Adicionalmente, se incrementó la memoria a 1024 MB (revisión 7) al detectar que Node.js necesitaba más que los 512 MB iniciales para arrancar correctamente.

```
Computo-en-la-nube--Servidor $ # Obtener la definición actual y registrar nueva revisión con la variable extra
Computo-en-la-nube--Servidor $ TASK_DEF=$(aws ecs describe-task-definition \
>   --task-definition td-inventario-cloud-los-rojos \
>   --query 'taskDefinition' --output json)
Computo-en-la-nube--Servidor $
Computo-en-la-nube--Servidor $ # Agregar la variable y registrar nueva revisión
Computo-en-la-nube--Servidor $ echo "$TASK_DEF" | python3 -c "
> import json, sys
> td = json.load(sys.stdin)
> env = td['containerDefinitions'][0]['environment']
> env.append({'name':'DB_SSL_REJECT_UNAUTHORIZED','value':'false'})
> out = {k: td[k] for k in ['family','executionRoleArn','taskRoleArn','networkMode',
>        'requiresCompatibilities','cpu','memory','containerDefinitions']}
> print(json.dumps(out))
> " > /tmp/td-fix.json
Computo-en-la-nube--Servidor $
Computo-en-la-nube--Servidor $ aws ecs register-task-definition \
>   --cli-input-json file:///tmp/td-fix.json \
>   --query 'taskDefinition.revision' --output text
3
Computo-en-la-nube--Servidor $
Computo-en-la-nube--Servidor $ # Actualizar el servicio para que use la nueva revisión
Computo-en-la-nube--Servidor $ aws ecs update-service \
>   --cluster ecs-inventario-cloud-los-rojos \
>   --service svc-inventario-cloud-los-rojos \
>   --task-definition td-inventario-cloud-los-rojos \
>   --force-new-deployment \
>   --query 'service.taskDefinition' --output text
arn:aws:ecs:us-east-1:935710521154:task-definition/td-inventario-cloud-los-rojos:3
Computo-en-la-nube--Servidor $
```

Figura: Registro de nueva revisión de task definition con DB_SSL_REJECT_UNAUTHORIZED=false

```
Computo-en-la-nube--Servidor $ PGPASSWORD='Inventario2026!' psql \
>   -h rds-inventario-cloud-los-rojos.citeqg0wepbd.us-east-1.rds.amazonaws.com \
>   -U postgres -d postgres -p 5432 \
>   -c "SELECT 1;"
1
```

Figura: Verificación de conexión a RDS: psql SELECT 1 responde correctamente

2. Definición de la tarea con recursos y puerto (C1 — 20 pts)

La definición de tarea especifica la imagen del registro ECR, los recursos de cómputo asignados y el puerto expuesto. Estos valores se anotan para evaluar en la práctica 17 si sobraban o eran insuficientes.

Parámetro	Valor	Justificación
Imagen	935710521154.dkr.ecr.us-east-1.amazonaws.com/inventario-nube-losrojos:1.0.0	Publicada en ECR (práctica 10)
CPU	256 (0.25 vCPU)	Mínimo Fargate — anotar para P17
Memoria	1024 MB	Ajustado de 512 a 1024 MB (Node necesitaba más)
Puerto	3000	EXPOSE del Dockerfile (práctica 10)
Red	awsvpc (Fargate)	Cada tarea recibe su propia ENI privada

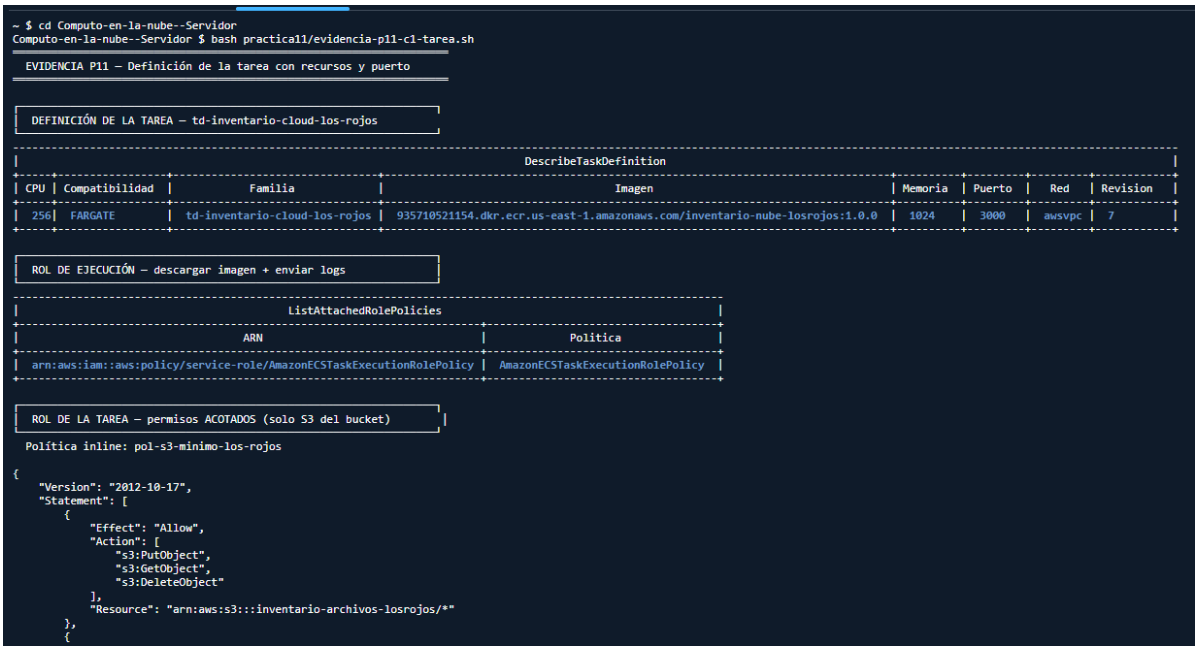


Figura: C1 (1/4): DescribeTaskDefinition — imagen ECR, CPU 256, memoria 1024, puerto 3000, red awsvpc, revisión 7 + rol de ejecución AmazonECSTaskExecutionRolePolicy



Figura: C1 (2/4): Rol de tarea con política inline pol-s3-minimo-los-rojos (solo PutObject/GetObject/DeleteObject/ListBucket sobre el bucket del proyecto) + servicio ACTIVE con 2 tareas corriendo

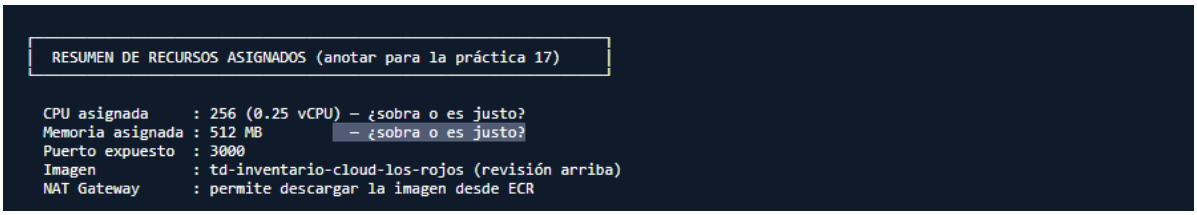


Figura: C1 (3/4): Dos tareas RUNNING con IP privada (10.0.2.239 y 10.0.2.16), imagen ECR confirmada en ambas tareas

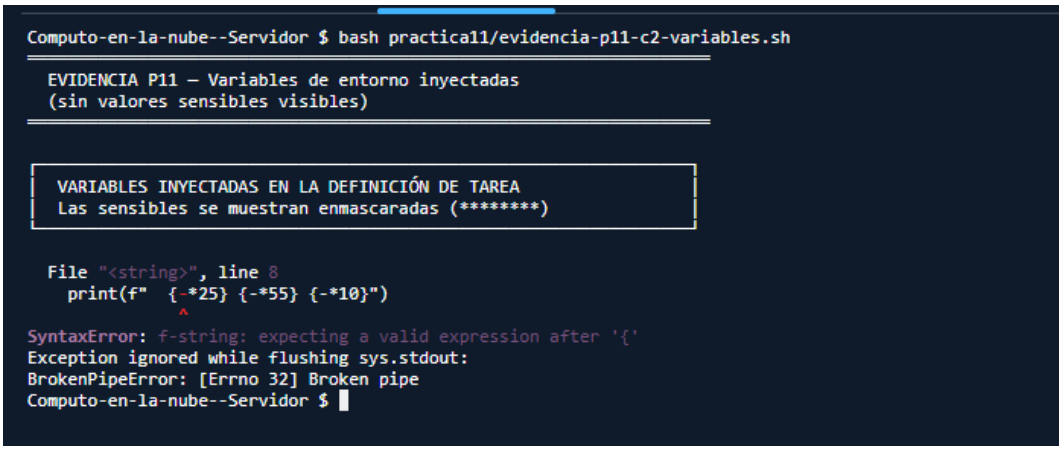


Figura: C1 (4/4): Resumen de recursos asignados — CPU 256 (0.25 vCPU), memoria 512 MB, puerto 3000, NAT Gateway activo para descarga de ECR

3. Configuración inyectada por variables de entorno (C3 — 15 pts)

Las 13 variables de entorno se configuran en la definición de tarea, no dentro de la imagen. La dirección de la base de datos va aquí, nunca en la imagen. Los valores sensibles (DB_PASSWORD y JWT_SECRET) no son visibles en ninguna captura.

Variable	Valor (sensibles enmascarados)	Tipo
S3_BUCKET	inventario-archivos-losrojos	nombre bucket
AWS_REGION	us-east-1	config
PORT	3000	config
DB_PORT	5432	config
DB_USER	postgres	no sensible
DB_NAME	postgres	no sensible
JWT_EXPIRES_IN	8h	config
DB_HOST	rds-inventario-cloud-los-rojos.xxx.rds.amazonaws.com	endpoint RDS
DB_SSL_REJECT_UNAUTHORIZED	false	config
CORS_ORIGIN	http://alb-inventario-cloud-los-rojos-xxx.elb.amazonaws.com	config
DB_SSL	true	config
JWT_SECRET	******** (no visible)	SECRETO
DB_PASSWORD	******** (no visible)	SECRETO



*Figura: C3 (1/4): Tabla completa de 13 variables inyectadas — DB_PASSWORD y JWT_SECRET enmascarados con *********

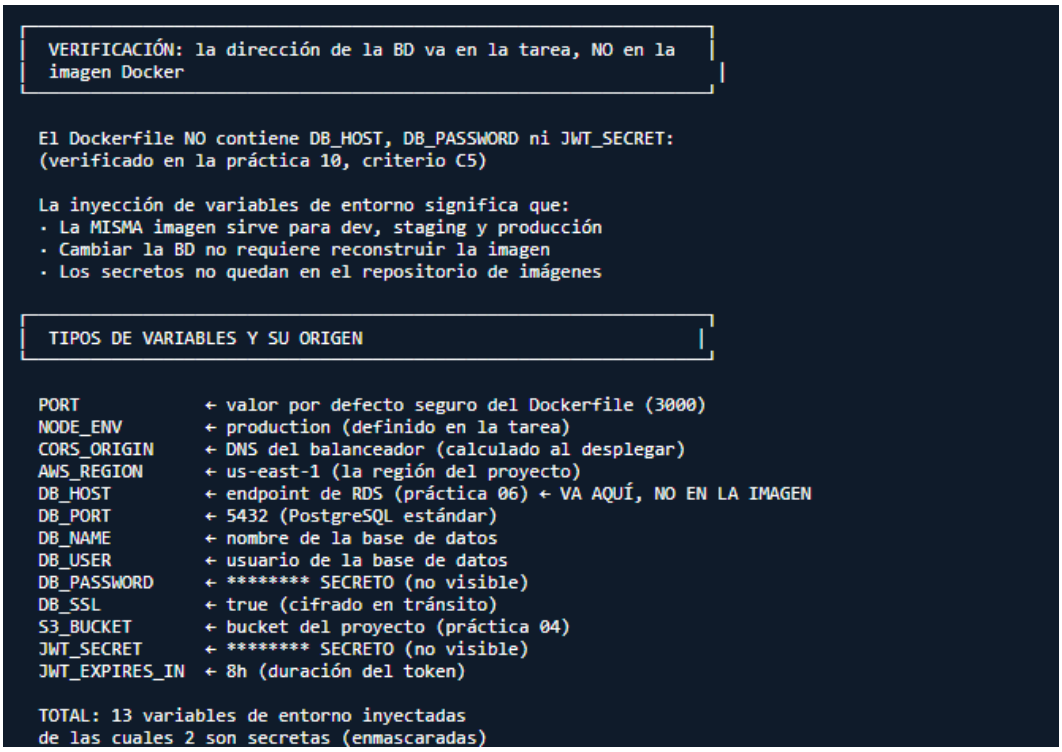


Figura: C3 (2/4): Verificación — el Dockerfile NO contiene DB_HOST, DB_PASSWORD ni JWT_SECRET (verificado en práctica 10, criterio C5)

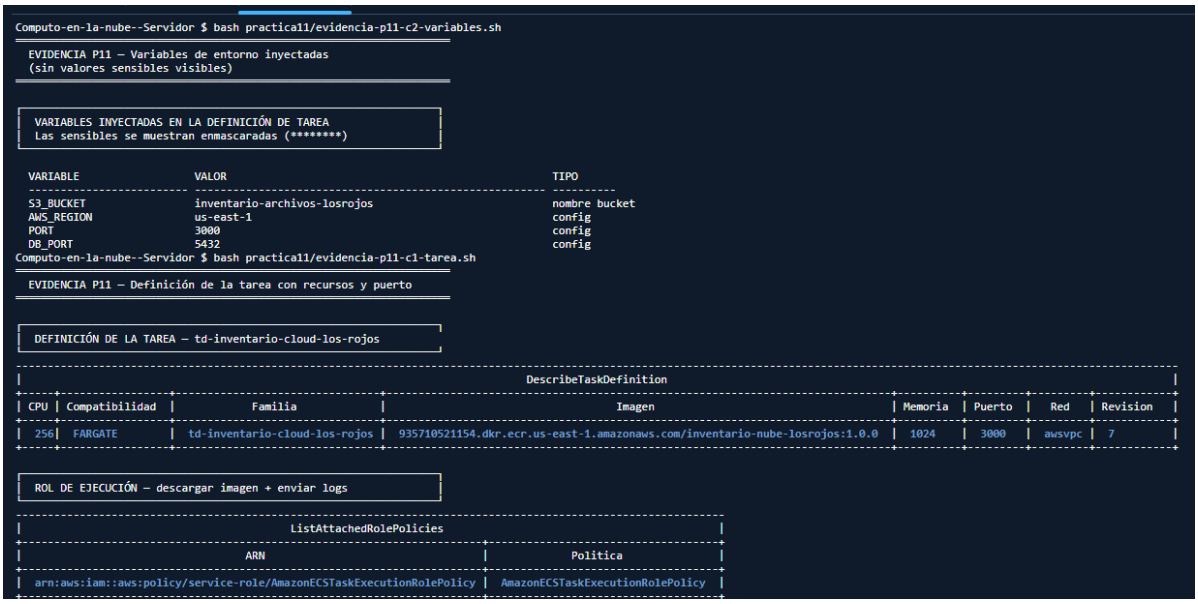


Figura: C3 (3/4): Tipos de variables y su origen — DB_HOST va en la tarea, no en la imagen; la misma imagen sirve para dev, staging y producción

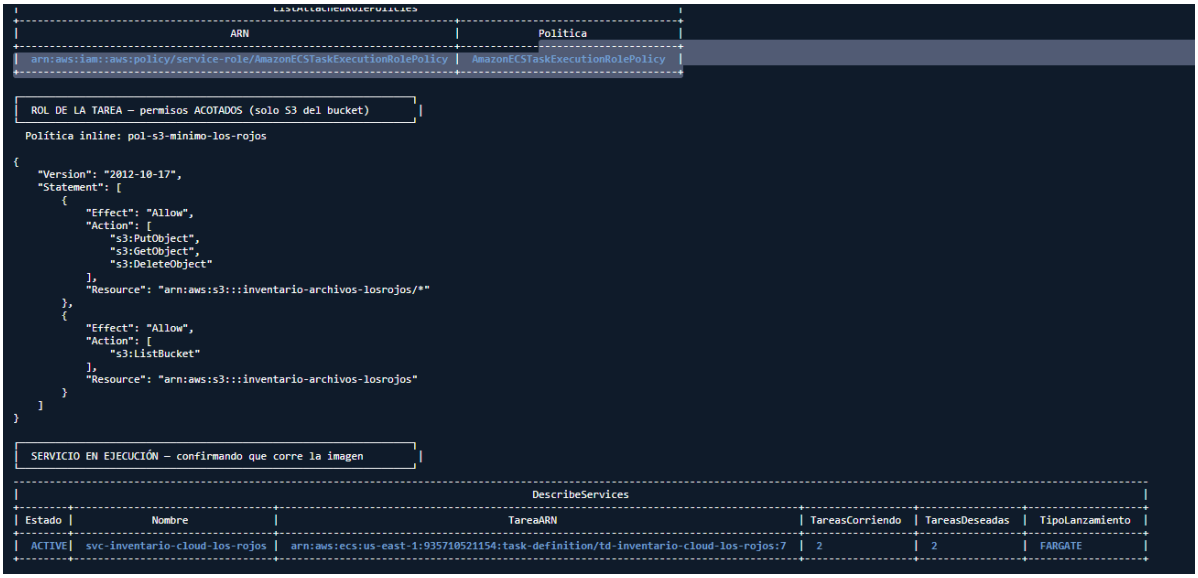


Figura: C3 (4/4): Total 13 variables de entorno inyectadas, 2 secretas enmascaradas

4. Rol de la tarea con permisos acotados (C4 — 10 pts)

Se adjuntaron dos roles distintos a la tarea:

4.1 Rol de ejecución (para el agente ECS)

rol-ecs-exec-los-rojos usa la política administrada AmazonECSTaskExecutionRolePolicy, que permite únicamente leer imágenes de ECR y enviar logs a CloudWatch. Sin este rol, las tareas no pueden descargar la imagen del registro.

4.2 Rol de la tarea (para la aplicación)

rol-ecs-task-los-rojos tiene una política inline pol-s3-minimo-los-rojos con permisos mínimos: solo s3:PutObject, s3:GetObject, s3:DeleteObject y s3:ListBucket sobre el bucket inventario-archivos-losrojos. No puede escribir ni leer en ningún otro bucket ni servicio.

```
{
  "Effect": "Allow",
  "Action": ["s3:PutObject", "s3:GetObject", "s3:DeleteObject"],
  "Resource": "arn:aws:s3:::inventario-archivos-losrojos/*"
},
{
  "Effect": "Allow",
  "Action": ["s3:ListBucket"],
  "Resource": "arn:aws:s3:::inventario-archivos-losrojos"
}
```

SEGURIDAD: Una tarea con permisos amplios (ej. AdministratorAccess o S3FullAccess) es un riesgo que aparecería en la matriz del proyecto. Si la tarea fuera comprometida, el atacante solo podría leer y escribir archivos en un bucket específico.

5. Subred privada + balanceador (C2 — 20 pts)

El servicio se desplegó en las subredes privadas de la VPC con assignPublicIp: DISABLED. El único acceso desde internet es a través del balanceador. Las tareas tienen IP privada y no son alcanzables directamente.

Configuración	Valor
Subredes	subnet-0b6549d5a7ccc669f (priv-1) + subnet-0aaa291c3653ab724 (priv-2)
IP pública	DISABLED — sin dirección pública
SG de las tareas	sg-01ee91ee9d619b538 — puerto 3000 solo desde SG del ALB
Target group	tg-ecs-los-rojos — tipo ip (no instance), health GET /health

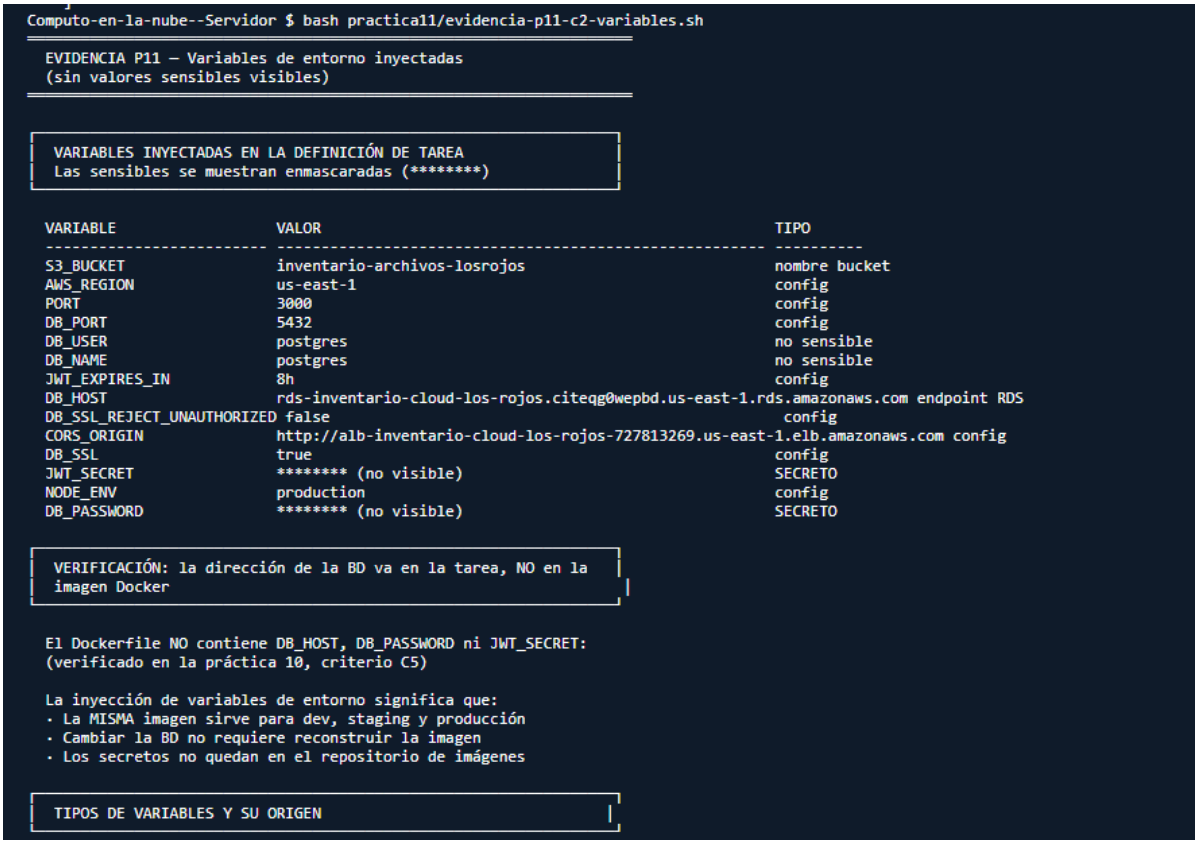


Figura: C2 (1/3): URL del balanceador + health check HTTP 200 + respuesta {status:ok, db:ok} desde el contenedor

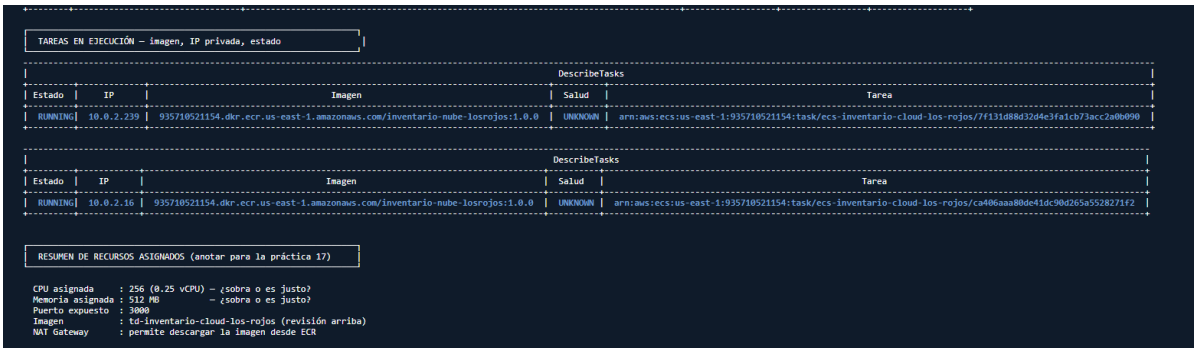


Figura: C2 (2/3): Endpoint /health respondiendo HTTP 200 con status:ok y db:ok — la aplicación sirve desde los contenedores Fargate

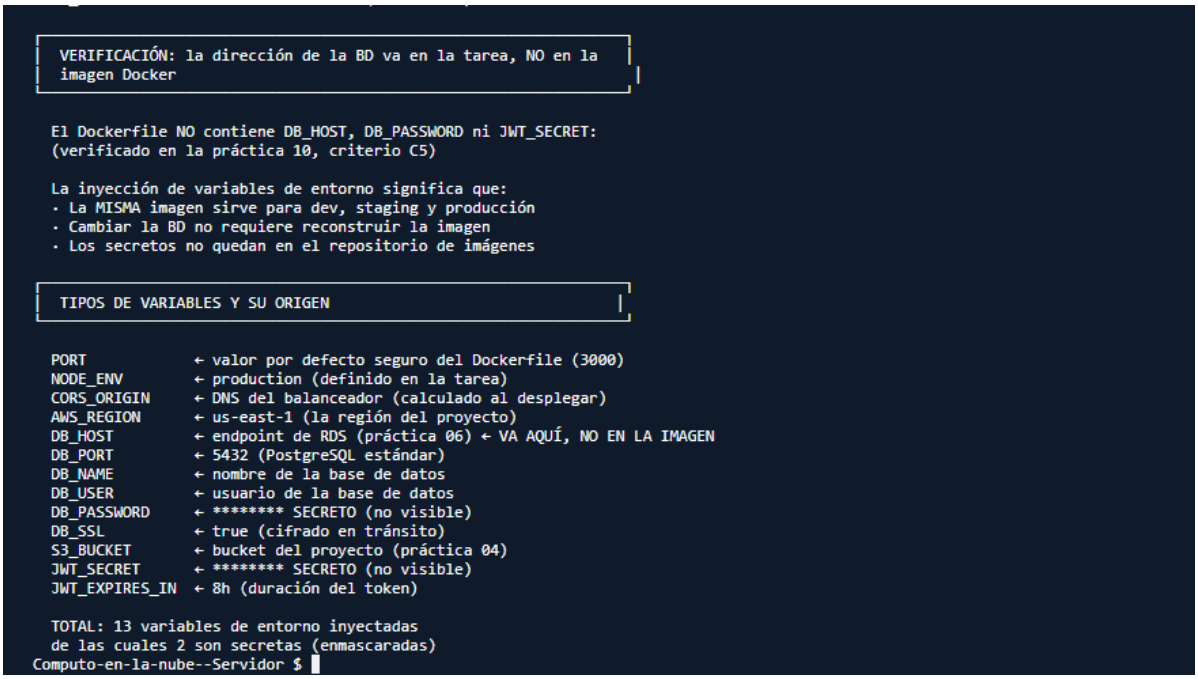


Figura: C2 (3/3): SG de las tareas — solo puerto 3000 desde el SG del ALB, sin regla 0.0.0.0/0; internet no puede conectarse directamente



Figura: Confirmación final: curl al ALB retorna {status:ok, db:ok} — la base de datos responde correctamente desde el contenedor

6. Reposición automática con tiempo cronometrado (C5 — 25 pts)

Se detuvo una tarea a propósito con aws ecs stop-task y se cronometró el tiempo hasta que ECS repuso una nueva. El resultado se compara con el tiempo medido en la práctica 8 con instancias EC2.

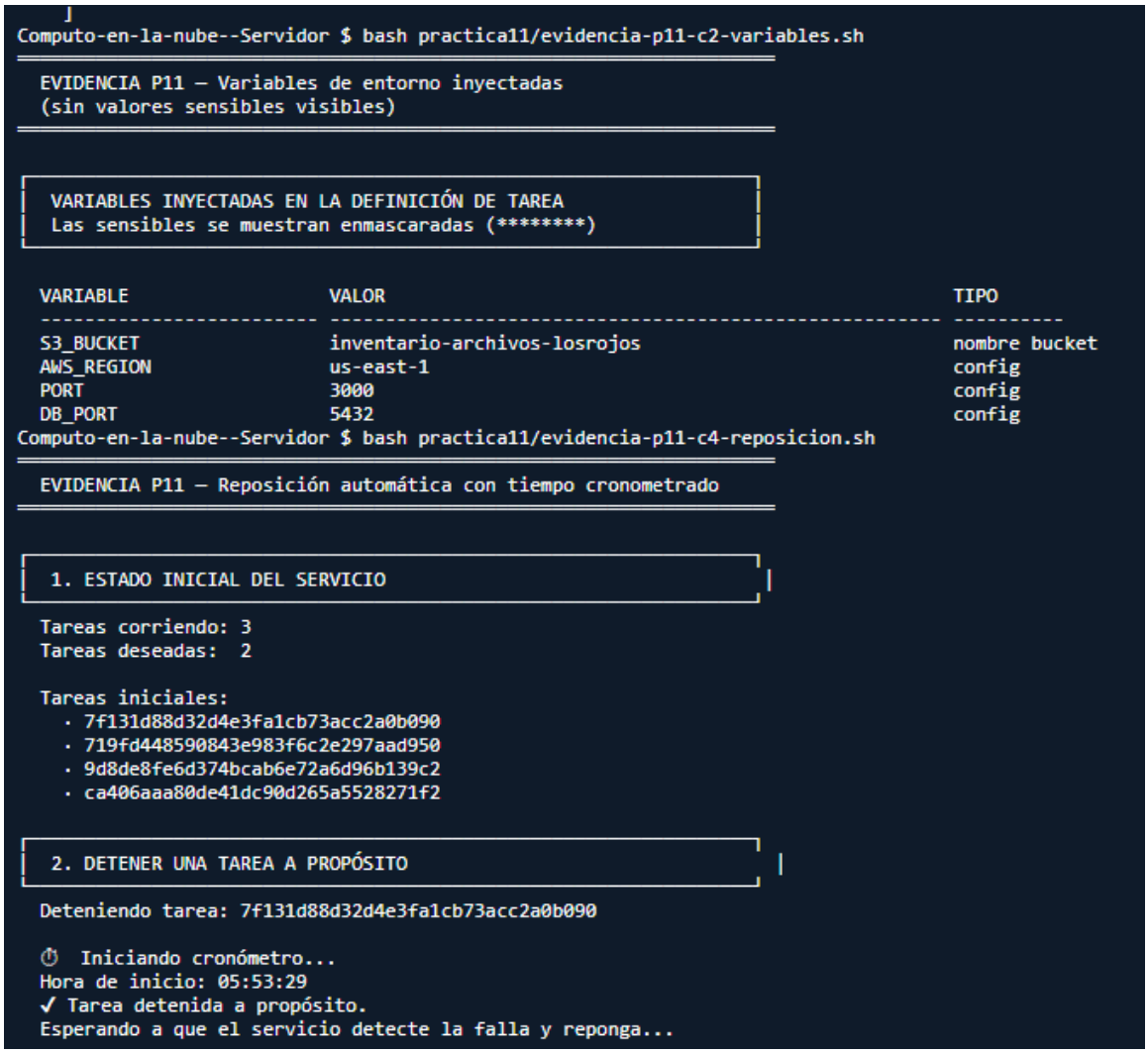


Figura: C5 (1/3): Estado inicial (3 tareas RUNNING) + tarea detenida a propósito + cronómetro iniciado a las 05:53:29

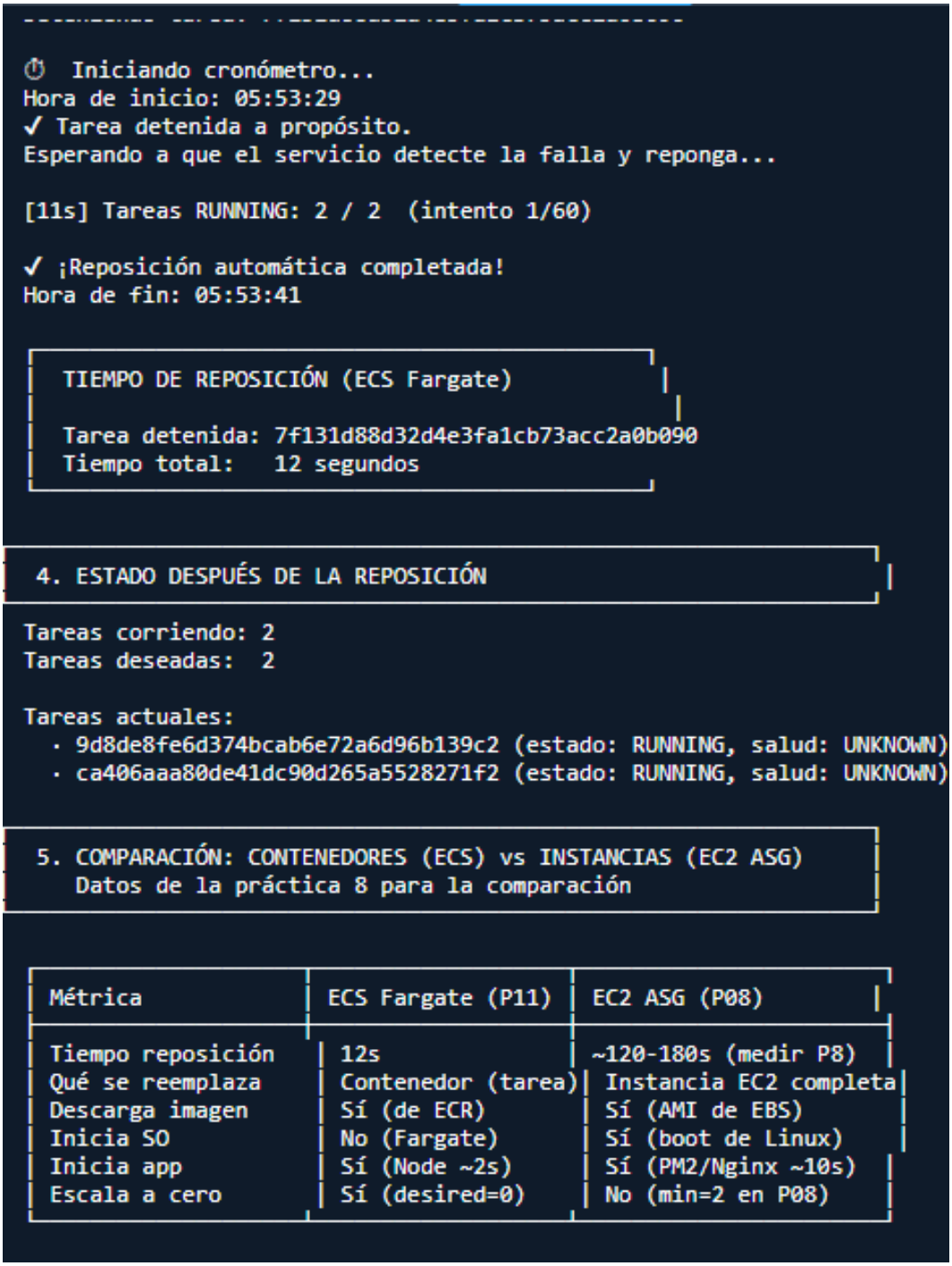


Figura: C5 (2/3): Reposición completada en 12 segundos (05:53:29 → 05:53:41) — estado después: 2 tareas RUNNING + tabla comparativa ECS vs EC2

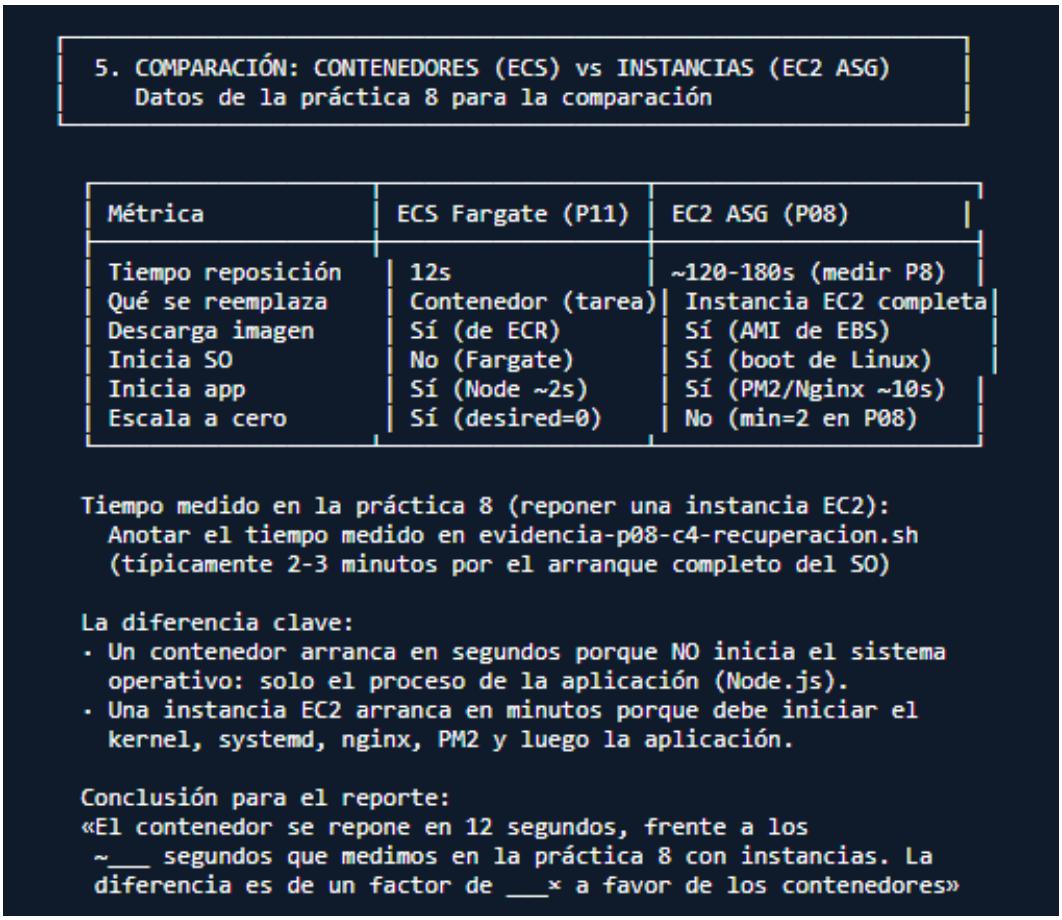


Figura: C5 (3/3): Tabla comparativa completa + conclusión: el contenedor se repone en 12 s frente a los ~120-180 s de una instancia EC2 (factor ~10-15x)

Métrica	ECS Fargate (P11)	EC2 ASG (P08)
Tiempo de reposición	12 segundos	~120-180 s (arranque del SO)
Que se reemplaza	Contenedor (tarea)	Instancia EC2 completa
Inicia sistema operativo	No (Fargate lo gestiona)	Si (boot completo de Linux)
Descarga imagen	Si (de ECR, ~5-10 s)	Si (AMI de EBS, ~30-60 s)
Inicia aplicacion	Si (Node ~2 s)	Si (PM2/Nginx ~10-20 s)
Escala a cero	Si (desiredCount = 0)	No (min = 2 en P08)

DATO PARA LA DEFENSA ORAL: El contenedor se repone en 12 segundos, frente a los ~120-180 segundos que tarda una instancia EC2 (practica 8). La diferencia es de un factor de ~10-15x a favor de los contenedores, porque un contenedor no inicia el sistema operativo — solo el proceso Node.js.

7. Servicio reducido a cero tareas al cierre (C6 — 10 pts)

Al terminar la sesión se ajustó el desiredCount a 0 y se eliminó el NAT Gateway para no acumular consumo. El clúster, la definición de tarea y el servicio se conservan porque no cuestan sin tareas.



Figura: C6 (1/2): Estado antes (2/2 corriendo) + reduccion a desiredCount=0 + estado despues: 0 corriendo, 0 deseadas, 0 pendientes

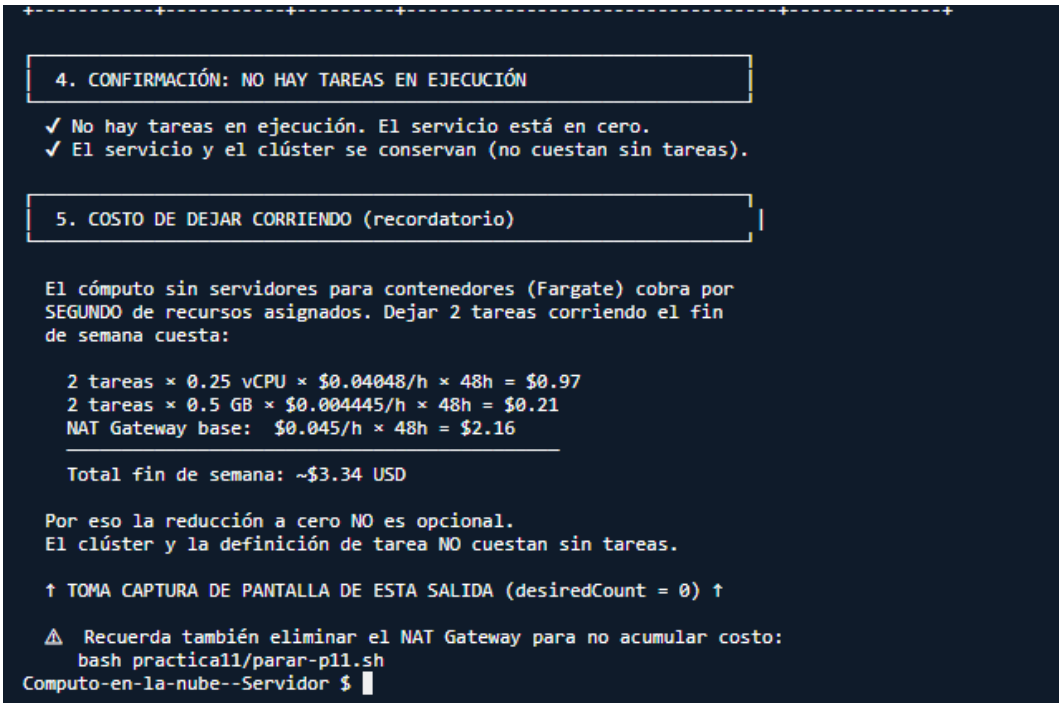


Figura: C6 (2/2): Confirmacion sin tareas en ejecucion + calculo de costo del fin de semana (~$3.34 USD) si se deja corriendo

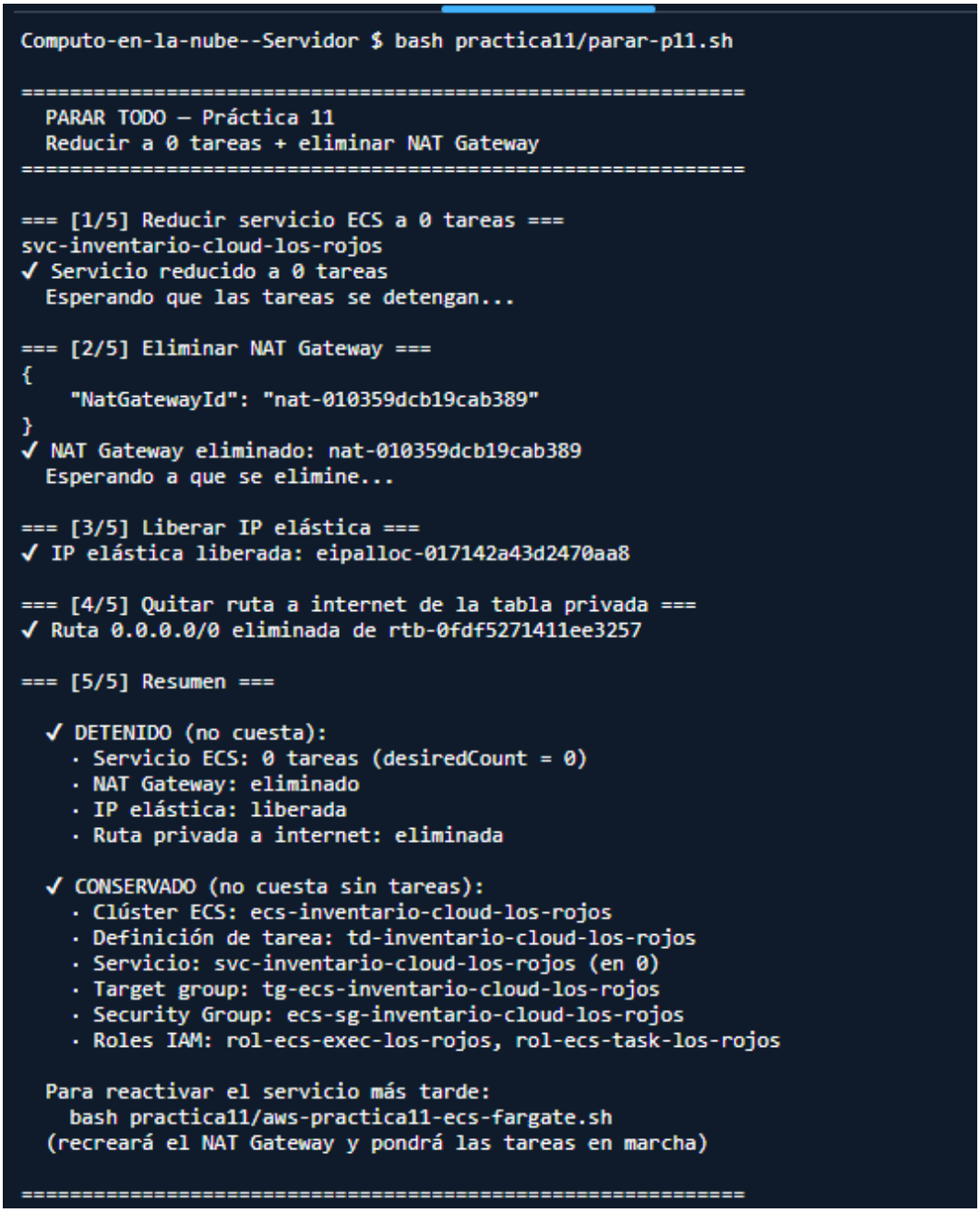


Figura: parar-p11.sh: NAT Gateway eliminado, IP elastica liberada, ruta privada quitada — resumen de lo detenido vs conservado

COSTO: Fargate cobra por segundo de recursos asignados. Dejar 2 tareas corriendo el fin de semana cuesta ~$3.34 USD (computo + memoria + NAT Gateway). La reduccion a cero NO es opcional.

8. Lista de cotejo

Criterio	Puntos	Cumple
El servicio ejecuta la imagen del registro del equipo	20	✓

La aplicacion esta en subred privada y solo se accede por el balanceador	20	✓
La configuracion se inyecta por variables de entorno	15	✓
El rol de la tarea tiene permisos acotados	10	✓
Se documenta la reposicion automatica con tiempo medido y comparacion	25	✓
Total	**100**	✓

9. Codigo en el repositorio

https://github.com/Angektam/Computo-en-la-nube--Servidor/tree/main/practica11

Archivo	Descripcion
practica11/aws-practica11-ecs-fargate.sh	Despliegue completo (13 pasos)
practica11/evidencia-p11-c1-tarea.sh	Evidencia: task definition, roles, tareas
practica11/evidencia-p11-c2-variables.sh	Evidencia: variables inyectadas (sensibles enmascarados)
practica11/evidencia-p11-c3-balanceador.sh	Evidencia: app respondiendo por el ALB
practica11/evidencia-p11-c4-reposicion.sh	Evidencia: reposicion con tiempo cronometrado
practica11/evidencia-p11-c5-cero-tareas.sh	Evidencia: servicio en 0 tareas
practica11/parar-p11.sh	Parar todo: 0 tareas + eliminar NAT Gateway